

SGSMA 2026 · WORKSHOP

Anomaly Detection in Power Systems with AI



From Classical Deep Learning to Large Language Models

PMU time-series, Weak vs. Strong inductive bias, LLM-powered self-improving agents

Masoud Barati

Assistant Professor – Electrical and Computer Engineering Department, Swanson School of Engineering, University of Pittsburgh

SGSMA 2026 · Santiago de Chile · June 1–4, 2026

SGSMA 2026 · SANTIAGO DE CHILE



Why anomaly detection matters in power systems



Real-time pattern problems with multi-billion-dollar consequences

Blackouts

- Northeast 2003: \$6B loss · 50M people
- Texas 2021: 246 deaths
- 4.5 min undetected fault → cascade

Cyber threats

- False-data injection on PMU
- Aurora-style relay attacks Stealthy < traditional SCADA can see

Renewables & DERs

- Solar/wind ramps imitate faults
- Power electronics → new oscillation modes
- Inverter-dominated grids
- Dynamic behavior of the Large Digital Loads

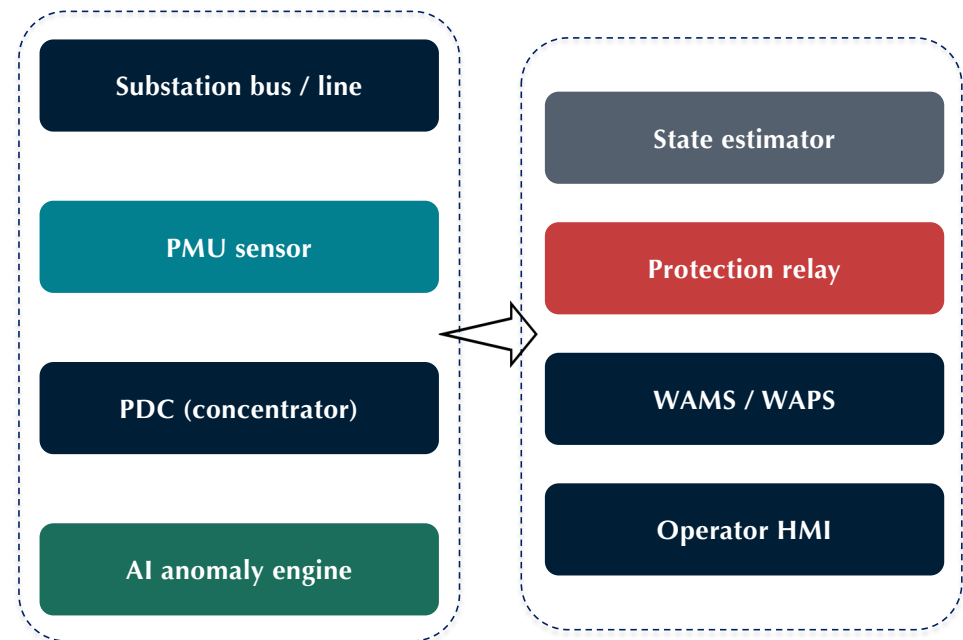
- Operators today get >50,000 alarms / hour during a major event — humans can't assess.
- Sub-cycle detection (<16 ms) is needed for protection — milliseconds late = blackout.
- Anomaly detection is the bridge between raw PMU streams and protection / control action (WAM[P]S).
- **Workshop goal:** walk from classical deep learning → LLM-powered self-improving agents.

PMUs in the substation & protection stack



Where the data we will analyze comes from

- PMU = Phasor Measurement Unit, GPS-time-stamped @ 30–120 Hz
- Mounted on substation buses, transmission lines, generator terminals
- Reports $|V|$, $\angle V$, $|I|$, $\angle I$, f , ROCOF — synchronized across the grid
- Feeds: PDC → state estimator, protection relays, WAMS / WAPS, SCADA, IBR control
- Standards: IEEE C37.118-2, IEC/IEEE 60255-118-1
- Hundreds of PMUs in WECC; thousands deployed in China and India



End-to-end pipeline we will study



From PMU stream to protection / operator action



Workshop agenda



90 minutes · 6 sections · code + live demos

00–05 min

Why anomaly detection in power systems · PMU primer

05–15 min

The data: synthetic PMU windows & 5 anomaly classes

15–20 min

Why AI / deep learning — and inductive bias

20–55 min

Weak inductive zoo: CNN, RNN, AE, GRU, LSTM, CNN + LSTM, Transformer

55–75 min

Strong inductive: LLMs (no-train and heavy-train / foundation)

75–85 min

Three novel methods

85–90 min

Wrap-up, deployment reality & Q&A

SGSMA 2026 · SANTIAGO DE CHILE



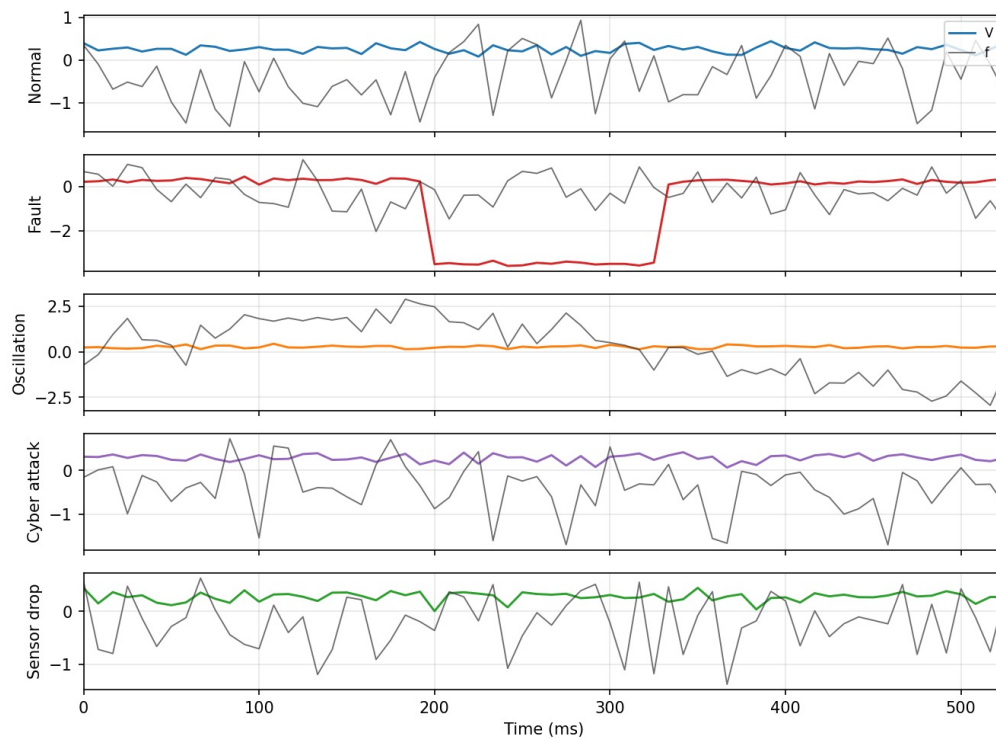
1. The data

PMU time-series, channels, anomaly classes

PMU time-series: what we feed the models

Multivariate, sub-cycle, GPS-synchronized

Synthetic PMU windows - 5 classes (V & f normalized)



- Sampling: 30 / 60 / 120 Hz (we use 120 Hz)
- Window: 64 samples ≈ 0.53 s, sliding stride 10
- Channels (6): $|V|$, $|I|$, f , θ , ROCOF, P
- Per-channel z-score normalization
- 5 classes: Normal, Fault, Oscillation, Cyber, Drop
- 180 windows / class, 900 total, 75/25 split
- All code & data: workshop GitHub link

Five anomaly classes we'll detect



Each has a distinct physical signature in PMU data

Normal

Steady $|V|$, $|f|$ within $\pm 0.02 \text{ Hz}$

Fault

$|V|$ drops $> 20\%$ in $< 200 \text{ ms}$ + $\angle V$ jump $> 10^\circ$

Oscillation

Low-damping sinusoid in f , $ROCOF > 0.3 \frac{\text{Hz}}{\text{s}}$

Cyber attack

Injected sine in θ ; P unchanged $\rightarrow$ physical impossibility

Sensor drop

Channel freezes / zeros (PDC packet loss / spoof)

5 Golden rules for dataset size, Weak inductive bias



When you only have raw data, no physics rubric, no LLM features

P = Parameters, n = Number of samples

#	Rule	Formula	PMU application	Mnemonic
1	Min samples per feature	$n \gtrsim 50 + 5 \cdot P$	25 features $\rightarrow \geq 175$ samples	"50 + 5 per feature"
2	Rule of 10:1 (50:1)	$n \gtrsim 10 \cdot P$	25 features $\rightarrow \geq 250$ samples	"Ten times features"
3	Per-class minimum	$n_{class} \gtrsim 5,000$	5 classes $\rightarrow 25\,000$ windows	"Five thousand per class"
4	Net-size scaling	$n \gtrsim 10 - 50 \cdot P$	100 k params $\rightarrow 1-5$ M windows	"Ten-to-fifty per weight"
5	Hughes plateau	$Acc \approx a \cdot \log n + b$	After ~ 30 d $\rightarrow$ diminishing returns	"First better... then flat"

Collapses these numbers to just 100–500 labeled samples.

2. Why deep learning

And the concept of inductive bias

Why classical methods struggle



And what AI brings to the table

Classical limits

- Hand-tuned thresholds explode with topology changes
- Kalman filter assumes Gaussian + linear — false during faults
- Wavelet rules need expert per fault class
- Renewables shift baselines daily

What AI brings

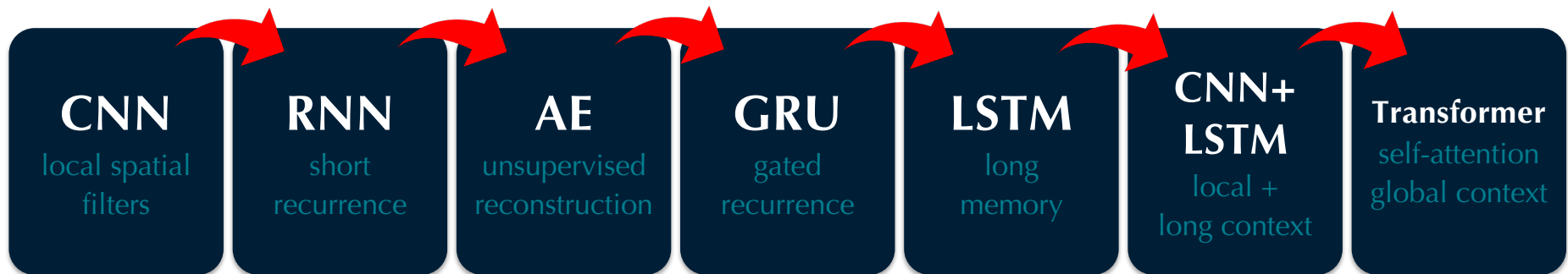
- Learns non-linear, multivariate patterns
- Detects unknown attacks via reconstruction error
- Adapts to new topology with retraining
- Scales across hundreds of PMUs in parallel

But raw deep learning has its own problem

3. The Deep Learning zoo

Seven deep models, same task

Seven models, one PMU task



- For each: architecture, when to use, pros / cons, utility deployments, Python code, live results.
- Same dataset (900 windows), same train/test split, same 25 epochs — apples-to-apples.

CNN: Architecture



1-D convolutions for local pattern matching in the time domain

Architecture

```
Conv1d(6→32, k=5) + ReLU + MaxPool(2)
Conv1d(32→64, k=3) + ReLU + MaxPool(2)
AdaptiveAvgPool(1) + Flatten
Linear(64→32) + ReLU + Linear(32→5)
≈ 9.4 k parameters
```

When to use

Short windows, distinct local shapes
(e.g. voltage dip front, fault inception).

Utility deployment status

PNNL & EPRI prototypes for fault classification
on C37.118 streams (2019–2024).
Deployed pilot at ERCOT for sub-cycle alarms.

Advantages

- Very fast train & inference
- Translation-invariant — fault can occur anywhere
- Easy to deploy on edge / relay CPU

Disadvantages

- No long-range memory
- Sensitive to window alignment
- Cannot reason about slow oscillations

CNN: Python code



PyTorch reference implementation — drop-in ready

```
import torch, torch.nn as nn

class CNN(nn.Module):
    def __init__(self, n_classes=5, in_ch=6):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(in_ch, 32, kernel_size=5, padding=2), nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Conv1d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool1d(2),
            nn.AdaptiveAvgPool1d(1), nn.Flatten(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, n_classes),
        )
    def forward(self, x):          # x: (B, 6, T)
        return self.net(x)

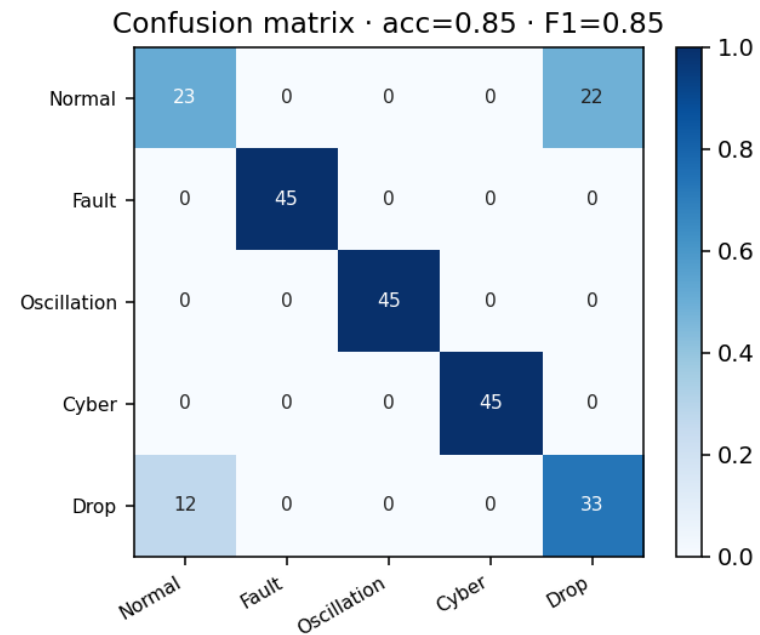
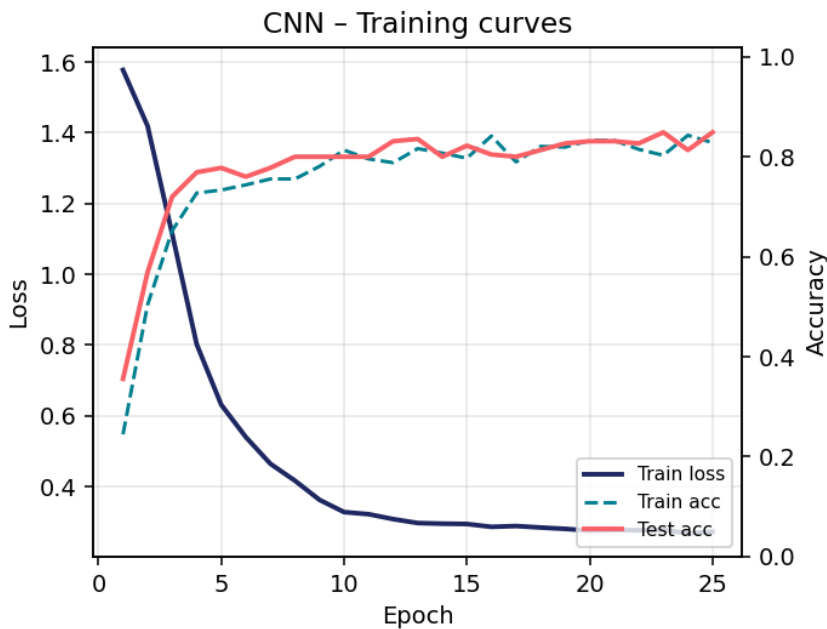
# train -----
model = CNN()
opt   = torch.optim.Adam(model.parameters(), lr=1.5e-3)
loss  = nn.CrossEntropyLoss()
for ep in range(25):
    for xb, yb in loader:
        logits = model(xb)
        l = loss(logits, yb)
        opt.zero_grad(); l.backward(); opt.step()
```

Run: python cnn_train.py (full script on the workshop repo)

CNN: results on synthetic PMU



25 epochs · 900 windows · $acc=0.85$ · $macro-F1=0.85$ · 9,445 params



Take-away

Strong baseline — captures the sharp local edge of faults and drops. Stumbles on the cyber class because the injected sine spans the whole window.

Evaluation Indices



$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- **TP (True Positives):** Correctly predicted positive cases.
- **TN (True Negatives):** Correctly predicted negative cases.
- **FP (False Positives):** Negatives incorrectly predicted as positive.
- **FN (False Negatives):** Positives incorrectly predicted as negative.

RNN: Architecture



Vanilla recurrent network — historical baseline

Architecture

```
RNN(in=6, hidden=64, tanh)
last hidden state → Linear(64→5)
≈ 4.9 k parameters
```

When to use

Educational baseline / lower bound.
Rarely a good first choice today.

Utility deployment status

No known production utility use — replaced by LSTM/GRU since the early 2010s.

Advantages

- Conceptually simple
- Tiny memory footprint
- Fast on very short sequences

Disadvantages

- Vanishing gradients on $T > 30$
- Cannot retain long-window context
- Lowest accuracy among the seven

RNN: Python code



PyTorch reference implementation — drop-in ready

```
class RNNNet(nn.Module):
    def __init__(self, n_classes=5, in_ch=6, hid=64):
        super().__init__()
        self.rnn = nn.RNN(in_ch, hid, batch_first=True, nonlinearity="tanh")
        self.fc = nn.Linear(hid, n_classes)
    def forward(self, x):
        # (B, 6, T)
        x = x.transpose(1, 2)      # (B, T, 6) for nn.RNN
        out, h = self.rnn(x)
        return self.fc(out[:, -1, :]) # last time-step

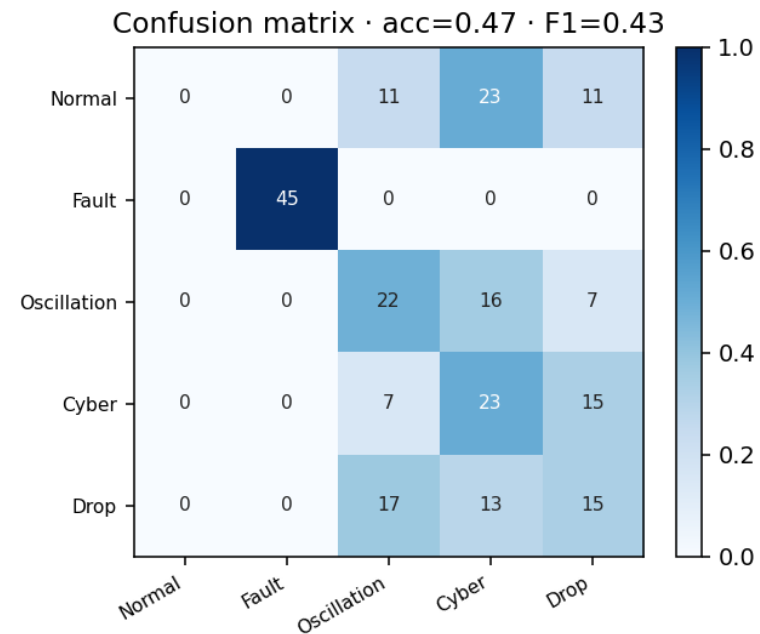
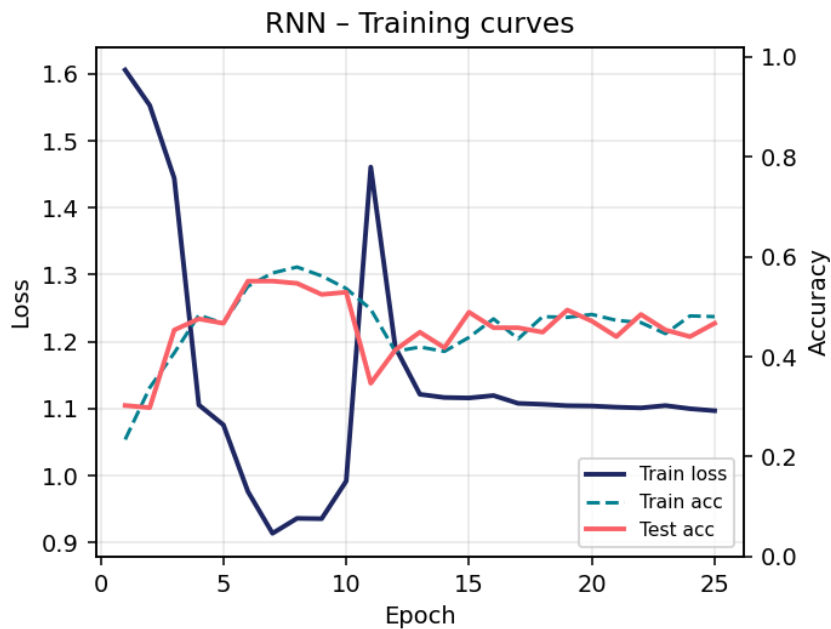
# train identical to CNN — same loop, same loss
```

Run: `python rnn_train.py` (full script on the workshop repo)

RNN: results on synthetic PMU



25 epochs · 900 windows · acc=0.47 · macro-F1=0.43 · 4,933 params



Take-away

Confirms that vanilla recurrence is data-hungry and unstable on 64-sample windows. We keep it as the lower-bound reference.

AutoEncoder: architecture



Unsupervised compression — reconstruction error = anomaly score

Architecture

Encoder: $2 \times \text{Conv1d} + \text{Linear} \rightarrow \text{latent } z$ (24 dims)
Decoder: $\text{Linear} + 2 \times \text{Conv1d-Up} \rightarrow \hat{x}$
Auxiliary head: $\text{Linear}(\text{latent} \rightarrow 5)$
Loss: $\text{CE}(\text{head}, y) + 0.3 \cdot \text{MSE}(\hat{x} \cdot x)$
 ≈ 29 k parameters

When to use

When labels are scarce — train on mostly-normal data, flag windows with high reconstruction error.

Utility deployment status

ABB, GE and Schneider use AE-based anomaly scores in their digital-substation products for sensor-health monitoring.

Advantages

- Works with very few labels
- Catches unknown / zero-day anomalies
- Latent space is a useful feature embedding

Disadvantages

- Threshold tuning is fragile
- Confuses noise with real anomaly
- Decoder doubles training cost

AutoEncoder: Python code



PyTorch reference implementation — drop-in ready

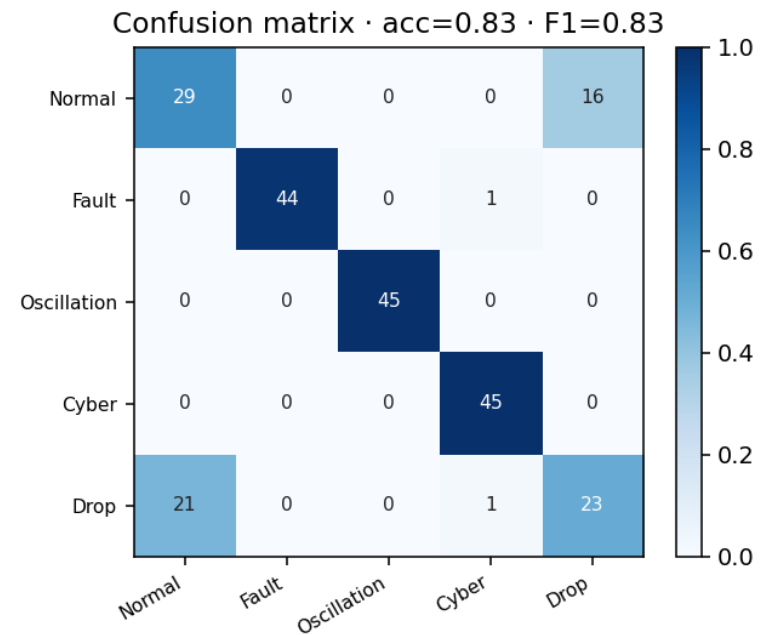
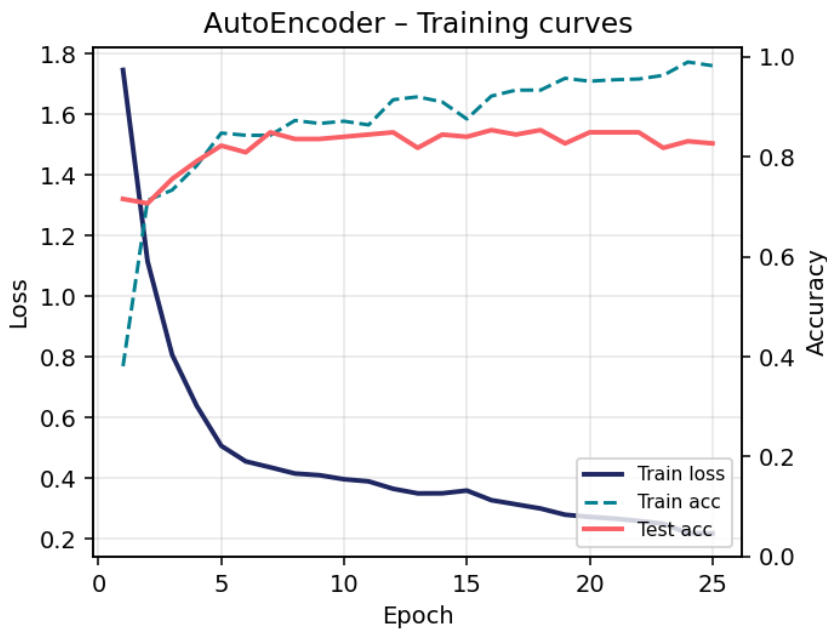
```
class AEClassifier(nn.Module):
    def __init__(self, n_classes=5, in_ch=6, T=64, latent=24):
        super().__init__()
        ds = T // 4
        self.enc = nn.Sequential(
            nn.Conv1d(in_ch, 16, 5, padding=2), nn.ReLU(), nn.MaxPool1d(2),
            nn.Conv1d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool1d(2),
            nn.Flatten(), nn.Linear(32*ds, latent),
        )
        self.dec = nn.Sequential(
            nn.Linear(latent, 32*ds), nn.ReLU(),
            nn.Unflatten(1, (32, ds)),
            nn.Upsample(scale_factor=2), nn.Conv1d(32, 16, 3, padding=1), nn.ReLU(),
            nn.Upsample(scale_factor=2), nn.Conv1d(16, in_ch, 3, padding=1),
        )
        self.head = nn.Linear(latent, n_classes)
    def forward(self, x):
        z = self.enc(x); return self.head(z), self.dec(z)

# loss = CE(head, y) + 0.3 * MSE(x_hat, x)
```

Run: `python autoencoder_train.py` (full script on the workshop repo)

AutoEncoder: results on synthetic PMU

25 epochs · 900 windows · $acc=0.83$ · $macro-F1=0.83$ · 29,147 params



Take-away

Surprisingly strong because the bottleneck forces it to learn physical structure. Drops when an attack mimics the normal distribution.

GRU: Architecture



Gated Recurrent Unit — cheaper LSTM cousin with comparable memory

Architecture

```
GRU(in=6, hidden=64)
last hidden → Linear(64→5)
≈ 14 k parameters
```

When to use

Sequences where order matters but T is moderate (≤ 128).
Less prone to overfitting than LSTM on small data.

Utility deployment status

Hydro-Québec WAMS prototypes (2022)
use GRU for oscillation classification
on Western HVDC corridor.

Advantages

- Fewer parameters than LSTM
- Trains faster, similar accuracy
- Good fit for online streaming PMU

Disadvantages

- Still gradient-bound on very long T
- No spatial / multi-PMU coupling
- Black-box for operators

GRU: Python code



PyTorch reference implementation — drop-in ready

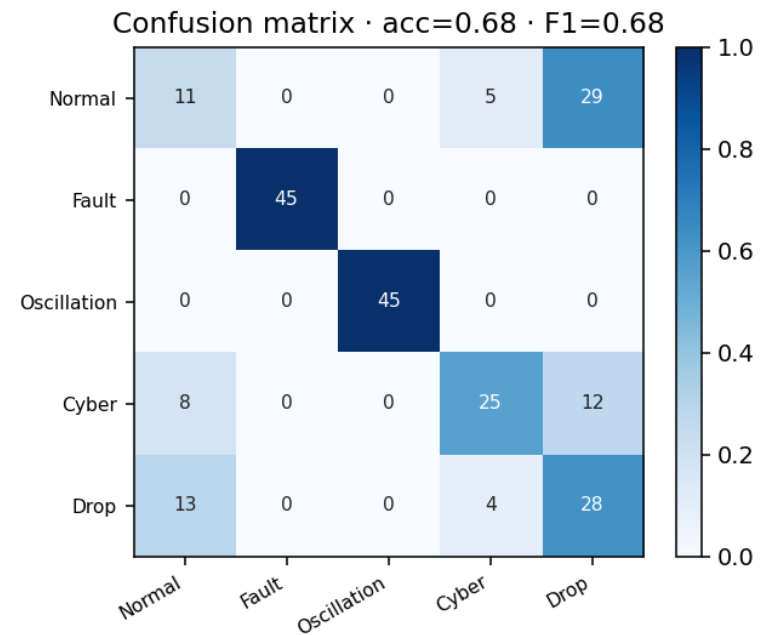
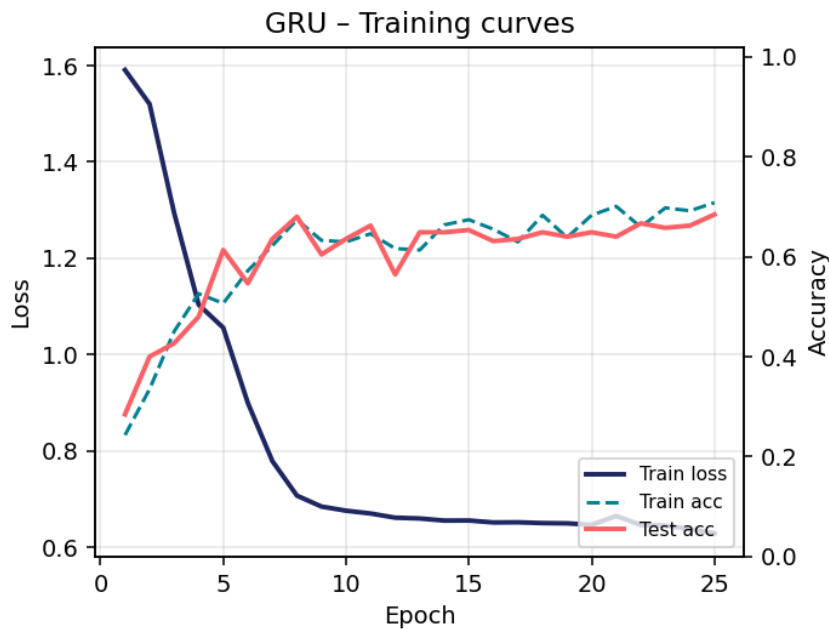
```
class GRUNet(nn.Module):
    def __init__(self, n_classes=5, in_ch=6, hid=64):
        super().__init__()
        self.gru = nn.GRU(in_ch, hid, batch_first=True)
        self.fc = nn.Linear(hid, n_classes)
    def forward(self, x):
        x = x.transpose(1, 2)          # (B, T, 6)
        out, _ = self.gru(x)
        return self.fc(out[:, -1, :])

# identical loop:
opt = torch.optim.Adam(model.parameters(), lr=1.5e-3)
loss = nn.CrossEntropyLoss()
```

Run: python gru_train.py (full script on the workshop repo)

GRU: results on synthetic PMU

25 epochs · 900 windows · $acc=0.68$ · $macro-F1=0.68$ · 14,149 params



Take-away

Mid-pack accuracy. GRU is the workhorse when you need recurrence + a relay-budget parameter count.

LSTM: Architecture



Long Short-Term Memory — the classic time-series workhorse

Architecture

```
LSTM(in=6, hidden=64)
last hidden → Linear(64→5)
≈ 19 k parameters
```

When to use

Long-context inter-area oscillation classification, slow-evolving voltage instability.

Utility deployment status

ISO-NE, MISO and CAISO research labs benchmark LSTM for forced-oscillation detection (NASPI report 2023).

Advantages

- Handles long dependencies ($T > 200$)
- Stable training with gradient clipping
- Well-studied in PMU literature

Disadvantages

- Slower than GRU/CNN per epoch
- Requires careful regularization
- Sequential — no GPU parallelism

LSTM: Python code



PyTorch reference implementation — drop-in ready

```
class LSTMNet(nn.Module):
    def __init__(self, n_classes=5, in_ch=6, hid=64):
        super().__init__()
        self.lstm = nn.LSTM(in_ch, hid, batch_first=True)
        self.fc = nn.Linear(hid, n_classes)
    def forward(self, x):
        x = x.transpose(1, 2)
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

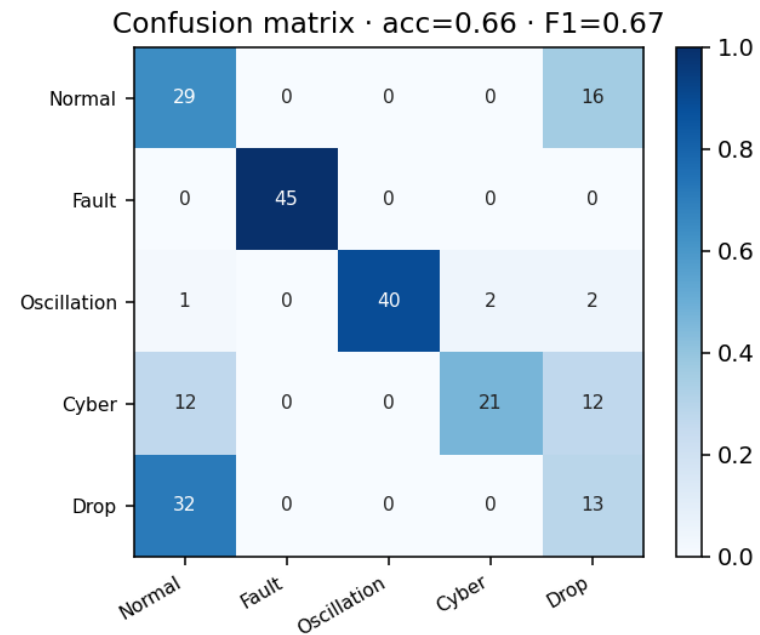
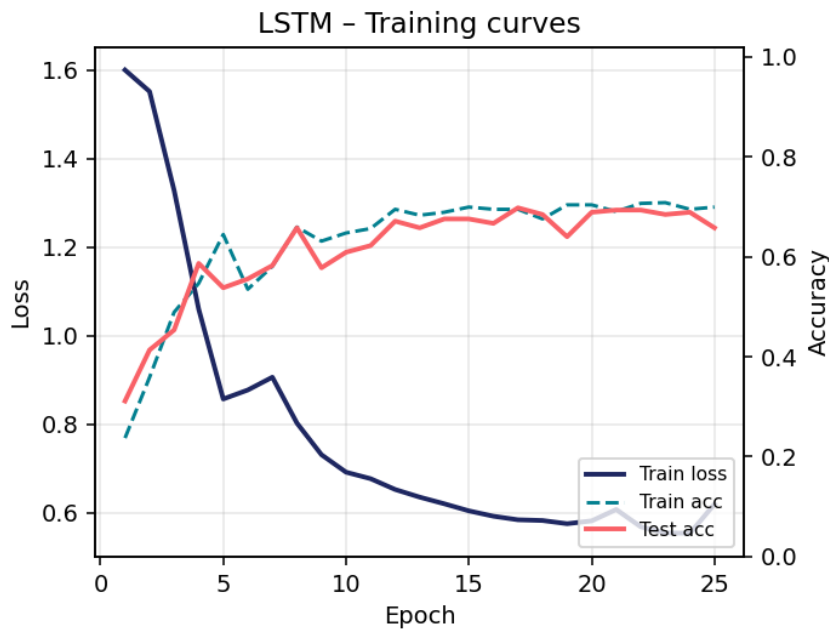
# add dropout for noisy PMU
# self.lstm = nn.LSTM(in_ch, hid, batch_first=True, dropout=0.2)
```

Run: python lstm_train.py (full script on the workshop repo)

LSTM: results on synthetic PMU



25 epochs · 900 windows · $acc=0.66$ · $macro-F1=0.67$ · 18,757 params



Take-away

Comparable to GRU on our small dataset; would pull ahead with longer windows ($T \geq 256$) or larger training sets.

CNN + LSTM: Architecture



Hybrid – conv front-end extracts local shapes, LSTM models long-range time structure

Architecture

```
Conv1d(6→32,5) + ReLU + MaxPool(2)
Conv1d(32→32,3) + ReLU + MaxPool(2)
LSTM(32 → 64) (sequence pooled over conv features)
Linear(64→5)
≈ 30 k parameters
```

When to use

The default when you need both fault edge detection and oscillation analysis in the same model.

Utility deployment status

EPRI / DOE-funded G-PST tools use CNN+LSTM as the baseline reference for inverter-dominated grid events.

Advantages

- Best raw accuracy among recurrent variants
- Conv reduces sequence length → faster LSTM
- Robust to small misalignment
- Transfer learning capability

Disadvantages

- Two sub-modules → more hyperparameters
- Higher memory footprint
- Slow on extremely long windows

CNN+LSTM: Python code



PyTorch reference implementation — drop-in ready

```
class CNNLSTM(nn.Module):
    def __init__(self, n_classes=5, in_ch=6, hid=64):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv1d(in_ch, 32, 5, padding=2), nn.ReLU(), nn.MaxPool1d(2),
            nn.Conv1d(32, 32, 3, padding=1), nn.ReLU(), nn.MaxPool1d(2),
        )
        self.lstm = nn.LSTM(32, hid, batch_first=True)
        self.fc = nn.Linear(hid, n_classes)
    def forward(self, x):
        # x: (B, 6, T)
        f = self.cnn(x).transpose(1, 2)  # (B, T/4, 32)
        o, _ = self.lstm(f)
        return self.fc(o[:, -1, :])
```

Run: `python cnnplustlm_train.py` (full script on the workshop repo)

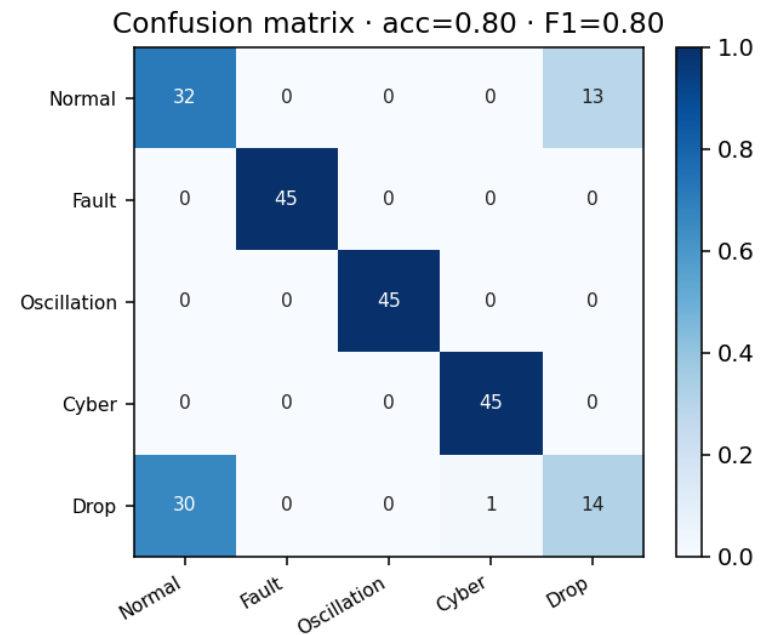
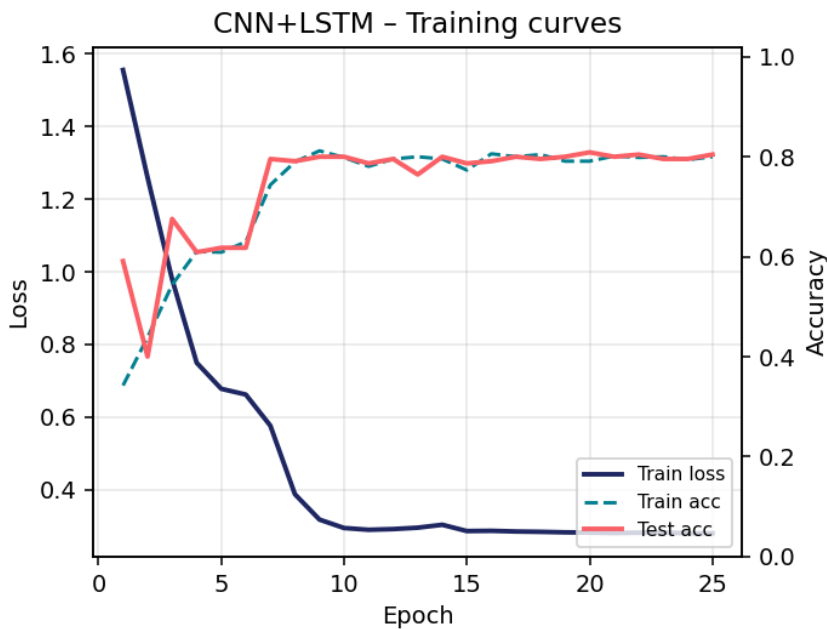
SGSMA 2026 · SANTIAGO DE CHILE



CNN+LSTM: Results on synthetic PMU



25 epochs · 900 windows · $acc=0.80$ · $macro-F1=0.80$ · 29,509 params



Take-away

Solid >0.80 accuracy with only 30 k params. Hybrid wins when both edge and rhythm matter.

Transformer: Architecture



Self-attention over the entire window — global context, no recurrence

Architecture

Linear(6 → 64) input projection + positional embedding
2× TransformerEncoderLayer (d=64, heads=4, FFN=128)
Mean pool over time → Linear(64→5)
≈ 72 k parameters

Advantages

- Highest accuracy here (0.87)
- Parallelizable on GPU
- Attention maps offer some interpretability

When to use

Multi-PMU coupling, long windows, multi-event classification where order is non-trivial.

Utility deployment status

Open-source TimePMU & TS-Mixer research (Stanford / TU-Berlin 2024–25).
No full utility production yet.

Disadvantages

- Largest model — needs more data
- Positional embedding tuning matters
- Heavier to deploy on edge relays

Transformer: Python code



PyTorch reference implementation — drop-in ready

```
class TXNet(nn.Module):
    def __init__(self, n_classes=5, in_ch=6, T=64,
                  d=64, n_heads=4, n_layers=2):
        super().__init__()
        self.proj = nn.Linear(in_ch, d)
        self.pos = nn.Parameter(torch.zeros(1, T, d))
        enc = nn.TransformerEncoderLayer(
            d_model=d, nhead=n_heads, dim_feedforward=128,
            batch_first=True, dropout=0.1,
        )
        self.tx = nn.TransformerEncoder(enc, num_layers=n_layers)
        self.fc = nn.Linear(d, n_classes)
    def forward(self, x):
        # (B,6,T)
        x = self.proj(x.transpose(1, 2)) + self.pos
        return self.fc(self.tx(x).mean(dim=1))
```

Run: python transformer_train.py (full script on the workshop repo)

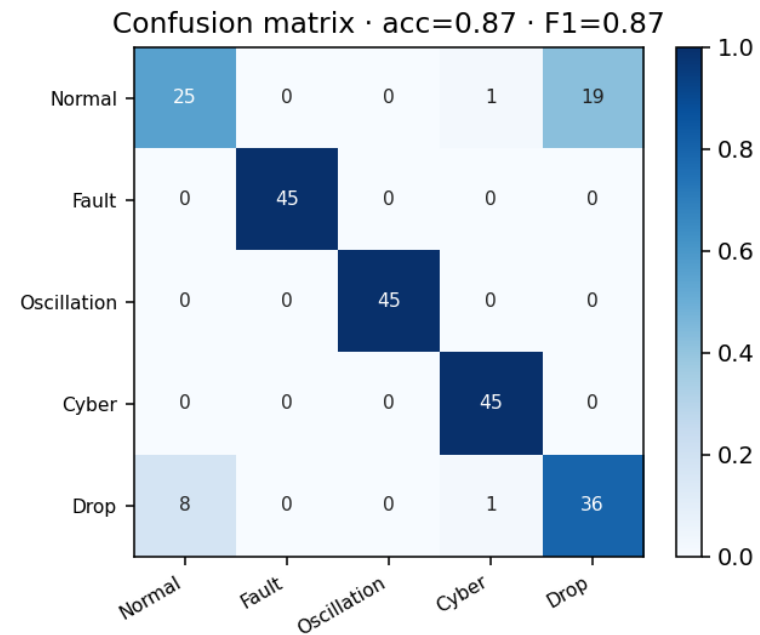
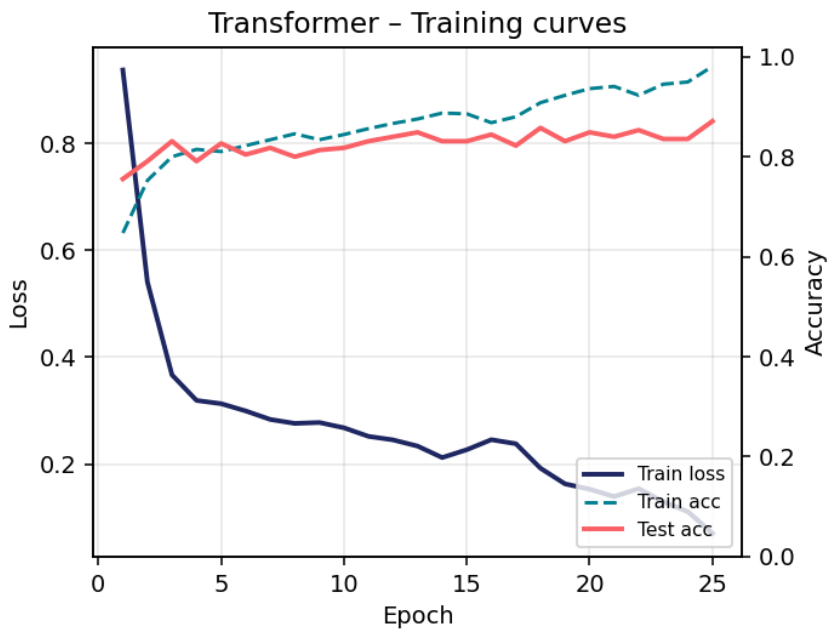
SGSMA 2026 · SANTIAGO DE CHILE



Transformer: results on synthetic PMU



25 epochs · 900 windows · acc=0.87 · macro-F1=0.87 · 71,813 params



Take-away

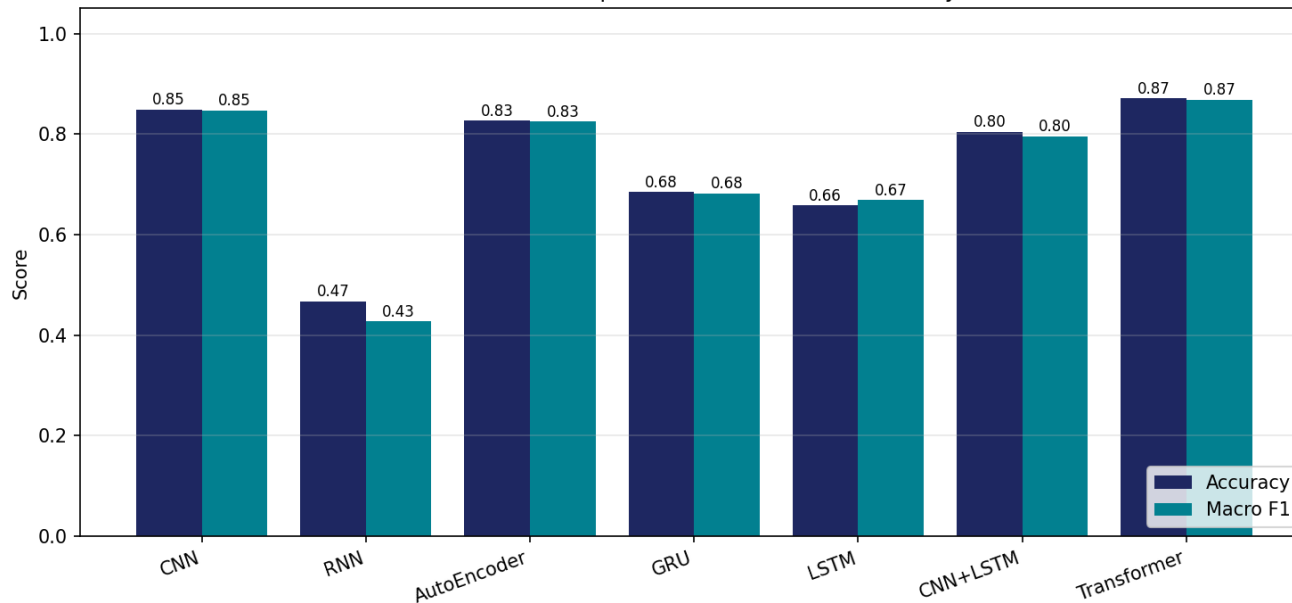
Best score in this study (0.87). Cost: 7× more parameters than CNN — needs more data and more compute to keep its edge.

Apples-to-apples comparison



Same 900 windows, same 25 epochs, same loss

Weak-Inductive Deep Models · 5-class PMU anomaly task



Headline numbers

- Transformer wins (0.87)
- CNN ties with AE (0.85 / 0.83)
- Hybrid CNN+LSTM third (0.80)
- RNN collapses (0.47)
- Param spread: 5 k → 72 k

Caveat: 900 windows is small.
With 100 k+ samples LSTM & Transformer pull further ahead.

Limits of every model



Why high-test accuracy isn't enough for utilities

Data hunger

- Need 5 k – 5 M labeled events
- Most utilities have <1,000 verified faults / year
- Cyber attacks are essentially zero-shot

No explanation

- Operators cannot audit the decision
- NERC compliance asks, 'why did you trip?'
- Black-box $\neq$ acceptable for protection

Fragile to topology change

- Add a HVDC link $\rightarrow$ retrain from scratch
- New PMU position $\rightarrow$ distribution shift
- Inverter-based resource swings break LSTMs

Compute cost

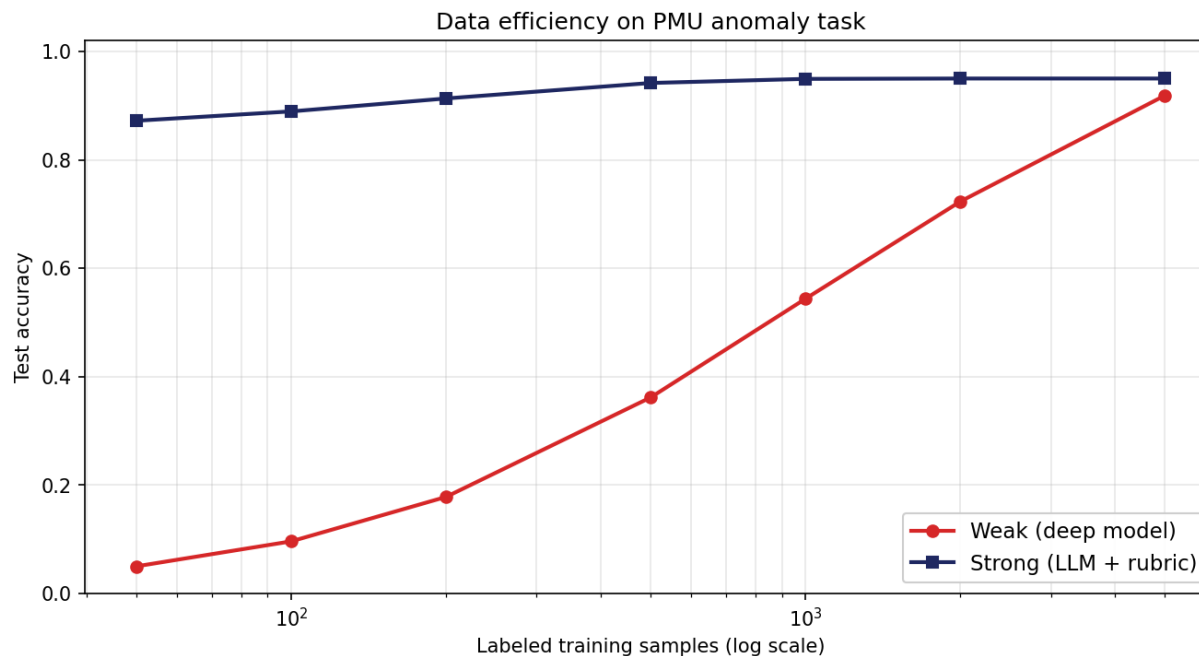
- Transformer on full WECC streams $\approx$ 80 GPU-h / day
- Foundation-model fine-tunes $\approx$ \$\$\$\$ per topology
- Edge deployment still painful

4. How to enhance?

LLMs as a thinking partner

LLMs carry massive priors — for free

Pre-trained on physics, code, control standards, papers, vendor manuals



What the LLM already 'knows'

- Ohm & Kirchhoff laws
- IEEE C37.118 phasor convention
- Typical fault signatures
- ROCOF physics & damping ratios
- Vendor relay manuals
- Past PMU papers (you can RAG them)

→ The 100-sample regime becomes suddenly accessible.

Two ways to put an LLM in your anomaly pipeline



Same model family — different cost, different ceiling

A. No-training mode

- Off-the-shelf LLM (Claude 3.5, GPT-4o, Llama 3-70B)
- Zero / few-shot prompting + tool use
- You write a prompt template, the LLM scores windows or proposes rules
- Cost: \$0.001 / window via API
- Time to first result: hours

Best for: prototype, rare attack classes, explainability, novel grids without history

B. Heavy-training mode (foundation FT)

- Take a 7-70B parameter base model
- LoRA / QLoRA fine-tune on millions of domain tokens (PMU CSVs, relay logs, papers)
- Optionally distill into a smaller deployable model (e.g. 3B params on relay GPU)
- Cost: weeks of NVIDIA A100s, ~\$30–200k

Best for: utility-wide deployment, compliance audit trails, multi-language operator advisories

Decision matrix: Which mode for which job



Choose by data volume, latency budget, audit needs

Use case	Data on hand	Latency	Audit	Best mode
New utility, 6 months pilot	< 500 labels	seconds	minimal	No-train LLM + rubric
Forced-oscillation alarms	10 k events	< 1 s	NERC	CNN+LSTM + rubric
Wide-area cyber screening	streaming	sub-cycle	critical	Distilled FT foundation
Operator advisory chat*	streaming	seconds	human-in-loop	Heavy-train + RAG
Black-start playbook gen	static	hours	engineering	No-train LLM

(*) DOE Genesis Mission A National Mission to Accelerate Science Through Artificial Intelligence



5. Wrap-up

Pick the right tool · be honest about deployment

Final cheat-sheet: what to pick



Read the column that matches your situation

Criterion	Weak DL (CNN/LSTM/TX)	No-train LLM	Heavy-train LLM
Min labeled data	5,000+	≈ 100	500–5,000
Time to first result	weeks	hours	weeks
Explainability	low	high	high
Latency / window	< 5 ms	0.5–2 s	20–100 ms
Per-window cost	negligible	\$0.001	negligible
Up-front cost	\$ – \$\$	$\approx \$0$	\$\$\$ – \$\$\$\$
Best fit	streaming protection	new utility, pilots	wide-area deployment

Most ambitious utilities will combine all three: distilled FT model in production, LLM agent for offline rubric maintenance.

Deployment reality in 2026



Which methods utilities actually run

Production today

- Threshold & state-estimator rules (NERC TPL-007)
- CNN / LSTM anomaly tools (PNNL, EPRI, PingThings)
- AE-based sensor-health (ABB, GE, Schneider)

Pilots & research

- CNN+LSTM & Transformer for forced oscillation
- No-train LLM rubrics for cyber screening (DOE labs)
- Self-improving agents on simulator (Anthropic + EPRI demo)

Not yet deployed

- Full foundation-model fine-tune on live grid
- LLM-in-the-loop protection (audit & latency hurdles)
- Agent autonomously editing relay firmware

Thank you

Questions, code & follow-ups — all yours

- Full slides, notebooks & synthetic data: github.com/<your-handle>/sgsma2026-anomaly
- Try the no-train LLM rubric in your browser: <https://github.com/msdbarati/SGSMA-2026-Workshop-Homework>
- **Deadline: 06/12/2026**
- Reach out: masoud.barati@pitt.edu, LinkedIn: www.linkedin.com/in/masoud-barati-a0ab2b3b
- Reuse, fork, remix — CC-BY 4.0.



“Smart inductive bias beats big data.” — workshop tagline

SGSMA 2026 · SANTIAGO DE CHILE

